

<https://plantuml.com/>

Sumário:

Jogo RPG Retrô Baseado em Estruturas Lineares e Dinâmicas

2. Completude dos Principais Requisitos

- 2.1 Objetivos Pedagógicos da Disciplina
- 2.2 Aplicação das Estruturas de Dados no Jogo
- 2.3 Justificativa do Uso de Estruturas Circulares, Listas, Filas e Pilhas

3. Concepção e Arquitetura do Sistema

- 3.1 Concepção e Levantamento de Requisitos
- 3.2 Arquitetura e Especificações Técnicas
- 3.3 Game Loop (Input $\rightarrow$ Update $\rightarrow$ Render)
- 3.4 Interfaces do Jogo
 - 3.4.1 Interface do Loop (Caminho e Cenário Adjacente)
 - 3.4.2 Interface de Combate Automático
 - 3.4.3 Interface de Inventário de Equipamentos e Mão de Cartas
 - 3.4.4 Interface de Menus e Telas do Jogo
- 3.5 Estruturas Internas para Caracterização do Jogo
 - 3.5.1 Lista Encadeada Circular Dupla (O Caminho do Loop)
 - 3.5.2 Matriz Estática Bidimensional (O Cenário e Terrenos Adjacentes)
 - 3.5.3 Fila Circular com Sentinela (Ordem de Turnos do Combate)
 - 3.5.4 Lista Encadeada Simples (Inventário de Itens/Equipamentos)
 - 3.5.5 Fila Dinâmica ou Vetor Limitado (Mão de Cartas do Jogador)
 - 3.5.6 Pilha Dinâmica (Gerenciamento de Menus e Telas do Sistema)
- 3.6 Comunicação entre Estruturas (Ex: Interação entre Nós da Lista Circular e Células da Matriz de Cenário)

4. Requisitos Funcionais (RF) (**pronto**)

5. Casos de Uso (CSU)

6. Requisitos Não Funcionais (RNF)

- 6.1 Restrições de Usabilidade e Interface de Texto/Gráfica
- 6.2 Restrições de Desempenho e Eficiência

- 6.2.1 Análise de Complexidade Algorítmica (Big-O de Inserção/Remoção no Loop e Inventário)
- 6.3 Restrições de Manutenibilidade e Gerenciamento de Memória
 - 6.3.1 Política de Prevenção de Vazamento de Memória (Desalocação dos Nós ao Fechar o Jogo)
- 6.4 Restrições de Plataforma de Hardware e Software

7. Identificação por TAG dos Artefatos

- 7.1 Padrão de TAGs do Projeto
- 7.2 Convenções de Nomeação de Variáveis e Funções

8. Rastreabilidade dos Artefatos

- 8.1 Rastreabilidade Vertical (Requisitos $\rightarrow$ Código)
- 8.2 Rastreabilidade Horizontal (Casos de Uso $\rightarrow$ Casos de Teste)

9. Tabela de Requisitos Funcionais (pronto)

10. Tabela de Casos de Uso (revisar)

11. Tabela de Casos de Teste (CT) (revisar)

12. Regras de Negócio (RN)

13. Matriz de Rastreabilidade

14. Especificações Técnicas (O Esqueleto do Código)

- 14.1 Definição das Estruturas dos Nós (Structs)
 - 14.1.1 Nó do Caminho (`CaminhoNode` - Ponteiros Duplos e Circulares)
 - 14.1.2 Nó da Fila de Combate (`CombateNode`)
 - 14.1.3 Nó do Inventário de Itens (`ItemNode`)
 - 14.1.4 Nó da Pilha de Interface (`MenuNode`)
- 14.2 Protótipos das Funções Principais de Manipulação de Ponteiros
- 14.3 Arquitetura e Organização dos Arquivos Fonte (`.c`, `.h` ou classes)
- 14.4 Diretrizes para Alocação Dinâmica e Uso de Ponteiros

15. O Prompt Zero

16. Anexos

- 16.1 Histórico de Alterações do Documento
- 16.2 Diagrama de Estruturas de Dados (Representação Visual do Loop e Inventário)
- 16.3 Fluxograma do Game Loop Principal
- 16.4 Referências Bibliográficas

1. Introdução e Visão Geral (**pronto**)

O projeto Herói em Loop, desenvolvido por (*nomes dos integrantes*), é um jogo inspirado em Loop Hero e criado com o propósito de aplicar, de maneira prática, os conhecimentos adquiridos na disciplina de Estrutura de Dados. O sistema utiliza conceitos fundamentais da programação e da organização de dados para transformar conteúdos teóricos em mecânicas funcionais dentro de um ambiente interativo.

A proposta principal do projeto é unir entretenimento e aprendizado técnico, utilizando estruturas como listas circulares, listas duplamente encadeadas, pilhas e filas para representar elementos do jogo, como mapa, inventário, cartas, movimentação e gerenciamento de recursos. Dessa forma, cada mecânica presente no sistema possui relação direta com algum conceito estudado durante a disciplina.

O jogo será executado em ambiente terminal e contará com um sistema de exploração contínua, combate contra inimigos, coleta de equipamentos e utilização de cartas para reconstrução do mapa. O jogador deverá evoluir seu personagem ao longo da jornada enquanto enfrenta desafios progressivamente mais difíceis até alcançar o confronto contra o chefe final.

Além de representar os conceitos de estruturas de dados, o projeto também busca desenvolver habilidades importantes relacionadas ao desenvolvimento de software, como lógica de programação, modularização do código, organização de funções, manipulação de memória e implementação de algoritmos de busca e ordenação.

Outro ponto importante do sistema é sua ligação entre narrativa e mecânicas. O mundo do jogo foi destruído e suas memórias apagadas, fazendo com que o jogador reconstrua gradualmente os caminhos e cenários utilizando cartas obtidas durante a exploração. Essa reconstrução simboliza a recuperação das lembranças do mundo perdido, criando uma integração entre gameplay e história.

Por se tratar de um projeto acadêmico, o foco principal não está em gráficos avançados ou recursos online, mas sim na implementação correta das estruturas de dados e no funcionamento lógico do sistema. Assim, o projeto prioriza desempenho, organização estrutural e aplicação prática dos conteúdos estudados.

1.1 Objetivo do Sistema (**pronto**)

O objetivo principal do sistema é desenvolver um jogo em linguagem C capaz de demonstrar, de forma prática e funcional, a aplicação dos conteúdos aprendidos na disciplina de Estrutura de Dados.

O projeto busca utilizar diferentes tipos de estruturas para representar elementos internos do jogo, permitindo exemplificar operações de armazenamento, organização, busca, manipulação e ordenação de dados dentro de um sistema interativo. Dessa forma, conceitos vistos apenas de maneira teórica passam a ser aplicados em situações reais de desenvolvimento.

Entre os principais objetivos técnicos do sistema estão:

- Aplicar listas circulares na construção do mapa do jogo;
- Utilizar pilhas no gerenciamento de itens e inventário;
- Implementar listas duplamente encadeadas para o controle das cartas;
- Trabalhar manipulação dinâmica de memória em linguagem C;
- Demonstrar algoritmos de organização e gerenciamento de dados;
- Simular estruturas utilizadas em sistemas reais de software e jogos.

Além dos objetivos técnicos, o projeto também possui objetivos acadêmicos e práticos relacionados ao desenvolvimento dos integrantes da equipe, como:

- Melhorar a capacidade de resolução de problemas;
- Desenvolver raciocínio lógico;
- Praticar trabalho em equipe;
- Aprimorar conhecimentos em programação estruturada;
- Entender a relação entre teoria e aplicação prática;
- Desenvolver experiência na construção de projetos completos.

O sistema também busca criar uma experiência simples, porém funcional, permitindo que o jogador interaja com mecânicas de exploração, combate e progressão enquanto as estruturas de dados operam internamente de forma integrada ao funcionamento do jogo.

Assim, o projeto Herói em Loop não serve apenas como um jogo acadêmico, mas também como uma demonstração prática do uso eficiente de estruturas de dados dentro de um sistema computacional completo.

1.2 Escopo do Projeto **(pronto)**

O sistema contempla as seguintes funcionalidades:

- Utilização de lista circular com sentinela para representar o mapa principal;
- Utilização de pilha para gerenciamento e ordenação de itens obtidos;
- Utilização de lista duplamente encadeada para armazenamento e manipulação das cartas do jogador;
- Sistema de geração de inimigos durante o percurso;
- Sistema de escolha de classes para o herói;
- Sistema de combate básico;
- Sistema de equipamentos contendo armas, armaduras, escudos e anéis;
- Progressão da partida dividida em início, meio e fim;
- Sistema de uso de cartas para reconstrução do mapa;
- Aparição de chefe final após determinadas condições do jogo.

O sistema não contempla:

- Sistema de salvamento de progresso;
 - Modo multiplayer;
 - Interface gráfica;
 - Sistema online;
 - Banco de dados.
-

1.3 Ambientação e História **(pronto)**

O projeto se passa na mesma ambientação de Loop Hero, onde todas as memórias do mundo foram apagadas por uma entidade desconhecida. O herói desperta em um vazio cercado apenas por um caminho sem fim, sem lembranças sobre o mundo, sobre as pessoas ou até mesmo sobre si mesmo.

Durante sua jornada, o jogador percorre continuamente o mapa enfrentando inimigos e recuperando fragmentos de memória na forma de cartas. Essas cartas representam partes esquecidas do mundo e podem ser utilizadas para reconstruir caminhos, paisagens e estruturas ao redor do mapa.

Cada elemento colocado no cenário simboliza uma lembrança restaurada. Assim, a mecânica de construção do mapa está diretamente ligada à narrativa do jogo, fazendo com que o progresso do jogador represente também a reconstrução gradual daquele universo perdido.

1.4 Jornada do Herói **(pronto)**

A jornada do herói se desenvolve através da exploração contínua do mapa circular, enfrentando inimigos, adquirindo equipamentos e utilizando cartas para reconstruir o mundo ao seu redor.

O ciclo principal do jogo funciona da seguinte maneira:

1. O jogador inicia sua jornada escolhendo uma classe;
2. O herói percorre automaticamente o caminho do mapa;
3. Durante o percurso, inimigos aparecem para combate;
4. Ao derrotar inimigos, o jogador recebe itens e cartas;
5. As cartas podem ser utilizadas para modificar o mapa e fortalecer ou dificultar a jornada;
6. O herói continua evoluindo até cumprir as condições necessárias para o aparecimento do chefe final.

O chefe final será liberado após o jogador atingir uma condição específica, como:

- Utilizar uma quantidade determinada de cartas; ou
- Completar um número específico de voltas no mapa circular.

Após derrotar o chefe, a partida será encerrada com a conclusão da jornada. Caso o herói seja derrotado durante o percurso, o jogo termina em derrota

1.5 Ficha Técnica do Jogo **(pronto)**

Característica	Descrição
Nome do Projeto	Herói em Loop
Gênero	RPG / Estratégia
Plataforma	Terminal
Linguagem	C

Estruturas de Dados Utilizadas	Pilha, fila, lista circular e lista duplamente encadeada
--------------------------------	--

Sistema de Combate	Combate automático em ciclos contínuos
--------------------	--

Movimentação	Automática
--------------	------------

Sistema de Inventário	Equipamentos e itens
-----------------------	----------------------

Classes do Herói	Sim
------------------	-----

Inimigos	Sim
----------	-----

Chefe Final	Sim
-------------	-----

Sistema de Cartas	Sim
-------------------	-----

Mapa	Tamanho e formato fixos
------	-------------------------

Funcionamento do Loop	O jogador não precisa comandar manualmente movimentação ou batalhas
-----------------------	---

Salvamento	Não
------------	-----

Interface Gráfica	Não
-------------------	-----

Descrição Técnica

O projeto será executado inteiramente em ambiente terminal e desenvolvido na linguagem C, utilizando conceitos de programação estruturada e manipulação dinâmica de memória.

O funcionamento principal do jogo ocorrerá em ciclos contínuos automáticos, onde o herói percorrerá o mapa e enfrentará inimigos sem necessidade de comandos

constantes do jogador para movimentação ou combate. Dessa forma, o jogador terá foco maior nas decisões estratégicas, como gerenciamento de itens, utilização de cartas e evolução do personagem.

O mapa possuirá tamanho e formato fixos, diferentemente do sistema presente em *Loop Hero*, onde o caminho pode variar dinamicamente. Essa escolha foi realizada para facilitar a implementação da lista circular utilizada na estrutura do mapa e manter maior controle sobre a lógica do sistema.

O jogo contará com:

- Sistema de classes para o herói;
- Sistema de equipamentos;
- Sistema de geração de inimigos;
- Sistema de cartas para modificação do mapa;
- Progressão baseada em ciclos;
- Chefe final condicionado ao avanço da partida.

O foco principal do projeto está na aplicação prática das estruturas de dados dentro das mecânicas do jogo, priorizando organização lógica, eficiência estrutural e integração entre teoria e implementação prática.

2. Completude dos Principais Requisitos

- 2.1 Objetivos Pedagógicos da Disciplina:

2.1 Objetivos Pedagógicos da Disciplina

O desenvolvimento do jogo foi proposto com o objetivo de aplicar, de forma prática e interativa, os principais conceitos estudados na disciplina de Estrutura de Dados. A proposta busca transformar conceitos teóricos em funcionalidades reais dentro de um sistema de jogo inspirado em *Loop Hero*, permitindo que os alunos visualizem o comportamento das estruturas em tempo de execução.

O projeto utiliza técnicas de alocação dinâmica de memória para criar e remover elementos do jogo durante a execução, simulando situações reais de gerenciamento de dados em aplicações computacionais. Dessa forma, o sistema trabalha constantemente com criação, manipulação e liberação de memória através de ponteiros, reforçando conceitos fundamentais da programação em linguagem C.

Além disso, o jogo permite a reutilização de estruturas de dados em diferentes sistemas internos, como movimentação do personagem, gerenciamento de inventário, controle de inimigos e manipulação de cartas e recompensas. Cada

mecânica foi planejada para representar um problema computacional real, utilizando a estrutura de dados mais adequada para sua solução.

Os principais objetivos pedagógicos do projeto são:

- Aplicar estruturas de dados lineares em um projeto funcional;
- Utilizar alocação dinâmica de memória em tempo de execução;
- Trabalhar manipulação de ponteiros e encadeamento de nós;
- Desenvolver lógica de inserção, remoção e busca de elementos;
- Demonstrar reutilização e organização eficiente de estruturas;
- Relacionar teoria e prática através do desenvolvimento de jogos;
- Simular cenários reais de gerenciamento de dados em sistemas computacionais.

Dessa forma, o jogo não funciona apenas como entretenimento, mas também como uma ferramenta prática de aprendizado, permitindo compreender de maneira visual e aplicada o funcionamento interno das estruturas de dados estudadas na disciplina.

2.2 Aplicação das Estruturas no Jogo

O jogo utiliza diferentes estruturas de dados para controlar suas mecânicas internas, permitindo que cada sistema funcione de maneira eficiente e organizada. Cada estrutura foi escolhida de acordo com a necessidade específica de funcionamento do jogo, simulando aplicações reais de gerenciamento de dados.

A pista principal do jogo é representada por uma Lista Circular com Sentinela. Essa estrutura controla o caminho percorrido automaticamente pelo herói, permitindo movimentação contínua em ciclos infinitos. O nó sentinela é utilizado para marcar o início e o fim lógico da pista, além de auxiliar no controle das voltas completas realizadas pelo personagem.

O inventário do jogador é controlado por uma Lista Simples Encadeada, responsável por armazenar equipamentos, itens e recursos coletados durante a exploração. Essa estrutura permite inserções e remoções dinâmicas sem necessidade de tamanho fixo previamente definido.

O sistema de cartas e recompensas utiliza uma Pilha, onde os elementos mais recentes obtidos pelo jogador podem ser utilizados primeiro. Essa abordagem representa o comportamento de “último a entrar, primeiro a sair” (LIFO), comum em mecânicas de armazenamento temporário.

Os inimigos e eventos automáticos do jogo são organizados através de uma Fila, permitindo que ações ocorram em ordem de chegada. Dessa forma, o sistema

mantém um fluxo organizado para spawn de inimigos, processamento de batalhas e execução de eventos do mapa.

As principais aplicações das estruturas no jogo são:

- Lista Circular com Sentinela:
 - Controle da pista/mapa do jogo;
 - Movimentação contínua do herói em ciclos;
 - Controle de voltas através do nó sentinela;
 - Organização dos blocos do caminho.
- Lista Simples Encadeada:
 - Gerenciamento do inventário;
 - Armazenamento de equipamentos e itens;
 - Inserção dinâmica de objetos coletados.
- Pilha:
 - Controle das cartas obtidas;
 - Armazenamento de recompensas temporárias;
 - Sistema de uso do último item adquirido primeiro.
- Fila:
 - Organização de inimigos e eventos;
 - Controle de spawn automático;
 - Processamento sequencial de ações do jogo.

Com isso, o projeto consegue integrar múltiplas estruturas de dados dentro de um único sistema, demonstrando como diferentes soluções podem trabalhar juntas para compor a lógica completa de um jogo digital.

2.3 Justificativa do Uso das Estruturas

As estruturas de dados utilizadas no desenvolvimento do jogo foram escolhidas com base nas necessidades técnicas de cada sistema interno. A seleção de cada estrutura busca garantir melhor organização dos dados, eficiência no processamento e compatibilidade com a lógica de funcionamento do jogo.

A Lista Circular com Sentinela foi escolhida para representar a pista principal do jogo devido ao funcionamento contínuo da movimentação do herói. Como o personagem percorre o mapa em ciclos infinitos, a estrutura circular permite que o último bloco esteja conectado novamente ao primeiro, eliminando a necessidade de reiniciar manualmente o percurso. O uso do nó sentinela facilita o controle lógico da estrutura, permitindo identificar quando uma volta completa foi concluída e simplificando operações de navegação no mapa.

A Lista Simples Encadeada foi utilizada no inventário porque a quantidade de itens do jogador pode variar constantemente durante a partida. Essa estrutura permite inserções e remoções dinâmicas sem necessidade de definir previamente um

tamanho máximo de armazenamento, além de reduzir desperdício de memória em comparação com vetores estáticos.

A Pilha foi escolhida para o sistema de cartas e recompensas devido ao comportamento natural da mecânica do jogo. Como os elementos mais recentes obtidos pelo jogador normalmente possuem prioridade de uso, a lógica “Último a Entrar, Primeiro a Sair” (LIFO) torna o gerenciamento mais simples e eficiente.

A Fila foi utilizada no controle de inimigos e eventos automáticos porque as ações do jogo precisam ocorrer em ordem de chegada. Dessa forma, o primeiro inimigo ou evento inserido no sistema será também o primeiro a ser processado, mantendo organização lógica durante o funcionamento do game loop.

As escolhas das estruturas também contribuem diretamente para os objetivos pedagógicos da disciplina, permitindo aplicar conceitos de:

- Alocação dinâmica de memória;
- Manipulação de ponteiros;
- Encadeamento de nós;
- Inserção e remoção dinâmica de dados;
- Organização eficiente de informações;
- Reutilização de estruturas computacionais.

Dessa forma, o projeto utiliza estruturas adequadas para cada situação do jogo, aproximando o desenvolvimento da aplicação de cenários reais encontrados no desenvolvimento de software e jogos digitais.

3. Concepção e Arquitetura do Sistema

3.1 Concepção e Levantamento de Requisitos

O projeto *Herói em Loop* foi concebido a partir da análise das principais mecânicas presentes no jogo Loop Hero, adaptando seus conceitos para um ambiente acadêmico voltado ao estudo de Estrutura de Dados em linguagem C.

Durante o processo de concepção, a equipe identificou quais mecânicas do jogo original poderiam ser representadas através de estruturas de dados estudadas em sala de aula. A partir dessa análise, foi realizado o levantamento dos requisitos funcionais e estruturais necessários para o funcionamento do sistema.

A primeira decisão de projeto foi definir que o mapa principal seria circular e contínuo, permitindo ao herói percorrer infinitamente o mesmo caminho até atingir as condições necessárias para enfrentar o chefe final. Essa característica levou à

escolha da Lista Circular Duplamente Encadeada como principal estrutura responsável pelo mapa.

Em seguida, foi identificado que o sistema precisaria controlar:

- movimentação automática do herói;
- gerenciamento de itens;
- armazenamento de cartas;
- geração de inimigos;
- sistema de combate;
- progressão do jogador;
- controle de equipamentos.

Com base nessas necessidades, foram definidos os seguintes requisitos estruturais:

- utilização de lista circular para o mapa;
- utilização de pilha para inventário e gerenciamento de itens;
- utilização de lista duplamente encadeada para cartas;
- utilização de filas para organização do combate;
- uso de alocação dinâmica de memória para criação de entidades durante a execução do jogo.

O projeto também foi planejado para funcionar inteiramente em ambiente terminal, priorizando lógica, desempenho e organização estrutural em vez de gráficos avançados. Dessa forma, os requisitos foram pensados para garantir simplicidade visual e foco total na aplicação prática das estruturas de dados.

Além disso, a narrativa do jogo foi integrada diretamente às mecânicas. O processo de reconstrução do mundo através das cartas representa simbolicamente a recuperação das memórias perdidas do universo do jogo, criando uma conexão entre gameplay e estrutura lógica do sistema.

cificações Técnicas: Visão geral de como o código se organiza (se é modular, dividido em funções, arquivos `.h` e `.c`, etc.).

3.2 Arquitetura e Especificações Técnicas

O sistema será desenvolvido utilizando programação estruturada em linguagem C, com organização modular do código para facilitar manutenção, reutilização e separação de responsabilidades.

A arquitetura do projeto será dividida em múltiplos arquivos `.c` e `.h`, permitindo que cada módulo do sistema seja responsável por uma funcionalidade específica.

A estrutura geral do projeto poderá ser organizada da seguinte forma:

/src

|

├── main.c

├── game.c

├── game.h

├── hero.c

├── hero.h

├── map.c

├── map.h

├── combat.c

├── combat.h

├── inventory.c

├── inventory.h

├── cards.c

├── cards.h

├── enemies.c

├── enemies.h

Cada módulo possuirá responsabilidades específicas:

Ar qui vo	Responsabilidade
--------------------------	-------------------------

ma in. c	Inicialização do jogo
----------------	-----------------------

ga me .c	Controle principal do sistema
----------------	-------------------------------

her o.c	Dados e ações do herói
------------	------------------------

ma p.c	Estrutura do mapa circular
-----------	----------------------------

co mb at. c	Sistema de combate
----------------------	--------------------

inv ent ory .c	Controle de itens e equipamentos
-------------------------	----------------------------------

car ds. c	Gerenciamento das cartas
-----------------	-----------------------------

en em ies .c	Criação e controle dos inimigos
-----------------------	------------------------------------

O projeto utilizará:

- ponteiros;
- alocação dinâmica de memória;
- structs;
- funções modulares;
- manipulação de listas;
- gerenciamento manual de memória.

A comunicação entre os módulos ocorrerá através de funções declaradas nos arquivos `.h`, garantindo encapsulamento e melhor organização do sistema.

A arquitetura foi planejada para facilitar:

- manutenção do código;
- expansão futura;
- reutilização de funções;
- separação lógica das mecânicas do jogo.

3.3 Game Loop

O funcionamento principal do jogo ocorrerá através de um loop contínuo responsável por manter toda a lógica da partida em execução.

A estrutura principal seguirá o modelo:

```
while(jogo_rodando)

{
```

```
input();  
  
update();  
  
render();  
  
}
```

O Game Loop será dividido em três etapas principais:

Input

Responsável por receber comandos do jogador.

Mesmo com movimentação automática, o jogador poderá:

- abrir inventário;
- utilizar cartas;
- equipar itens;
- visualizar status;
- realizar escolhas estratégicas.

Update

Responsável por atualizar o estado do jogo.

Nessa etapa ocorrerão:

- movimentação automática do herói;
- geração de inimigos;
- atualização da vida do jogador;
- processamento do combate;
- aplicação de efeitos;
- controle de progressão;
- verificação de derrota ou vitória.

Também será nesse momento que as estruturas de dados serão manipuladas diretamente.

Exemplo:

- avanço do ponteiro na lista circular;
- inserção de cartas;

- remoção de itens;
- atualização de filas de combate.

Render

Responsável pela exibição das informações no terminal.

Serão mostrados:

- mapa;
- posição do herói;
- inimigos;
- inventário;
- vida;
- equipamentos;
- cartas disponíveis;
- mensagens de combate.

O loop continuará em execução até:

- derrota do herói;
- vitória contra o chefe final;
- encerramento manual do jogo.

3.4 Interfaces do Jogo

3.4.1 Tela Inicial

A tela inicial será simples e exibida em terminal.

3.4.2 Tela de Gameplay

A tela principal exibirá o estado atual da partida.

3.4.3 Tela de Combate

Durante o combate, o terminal exibirá os dados dos personagens.

3.5 Estruturas Internas

3.5.1 Lista Circular Simples — Mapa

O mapa principal do jogo será implementado utilizando uma Lista Circular Simples. Cada nó da estrutura representará uma posição específica do caminho percorrido pelo herói durante a partida.

Cada posição do mapa poderá armazenar:

- tipo de terreno;
- inimigos;
- estruturas;
- eventos;
- modificações aplicadas por cartas.

A escolha da Lista Circular foi realizada porque o jogo possui movimentação contínua e repetitiva, permitindo que o herói percorra infinitamente o mesmo trajeto até cumprir as condições necessárias para enfrentar o chefe final.

A estrutura circular permite que o último nó do mapa aponte novamente para o primeiro nó, formando um ciclo contínuo de navegação.

O herói se movimentará apenas em uma direção dentro da lista, avançando continuamente para o próximo nó do percurso. Dessa forma, não existe necessidade de navegação reversa, tornando a Lista Circular Simples mais adequada para a lógica do sistema.

A utilização dessa estrutura permitirá:

- movimentação contínua automática;
- controle eficiente da posição do herói;
- repetição infinita do percurso;
- inserção lógica de eventos ao longo do caminho;
- representação prática do funcionamento do loop principal do jogo.

Cada nó da lista poderá conter ponteiros para:

- próximo elemento do mapa;
- inimigo presente;
- estruturas aplicadas;
- informações do terreno.

Exemplo conceitual da estrutura:

[Acampamento] → [Slime] → [Vazio] → [Goblin] → [Tesouro]



Assim, a Lista Circular Simples se torna a principal estrutura responsável pelo funcionamento do percurso automático do herói dentro do sistema.

3.5.2 Matriz do Cenário

Ao redor da pista circular existirá uma matriz representando o cenário visual e estruturas posicionadas pelas cartas.

A matriz permitirá:

- organização espacial;
- posicionamento de elementos;
- representação visual simplificada do mundo.

Ela coexistirá com a lista circular da seguinte maneira:

- a lista controla o caminho principal;
 - a matriz controla o ambiente externo ao redor do caminho.
-

3.5.3 Pilha — Inventário

O inventário utilizará uma Pilha.

Os itens mais recentes poderão ser acessados rapidamente através do topo da pilha.

A estrutura permitirá:

- inserção rápida;
 - remoção eficiente;
 - organização dinâmica dos equipamentos.
-

3.5.4 Lista Duplamente Encadeada — Cartas

As cartas serão armazenadas em uma Lista Duplamente Encadeada.

Isso permitirá:

- percorrer cartas em ambas direções;

- inserir novas cartas dinamicamente;
 - remover cartas utilizadas;
 - reorganizar facilmente o conjunto disponível.
-

3.5.5 Fila Circular — Combate

O sistema de combate utilizará Fila Circular para organizar turnos e ações.

A estrutura controlará:

- ordem de ataques;
 - entrada de inimigos;
 - processamento contínuo das ações.
-

3.5.6 Structs e Ponteiros

Todas as entidades do jogo serão representadas através de **structs**.

Exemplo:

- herói;
- inimigos;
- itens;
- cartas;
- posições do mapa.

Os ponteiros serão utilizados para:

- ligação entre nós;
 - manipulação dinâmica de memória;
 - navegação entre estruturas.
-

3.6 Comunicação entre Estruturas

As estruturas de dados do sistema funcionarão de maneira integrada.

A Lista Circular do mapa será responsável por controlar a posição atual do herói. Sempre que o ponteiro do jogador avançar para um novo nó, o sistema verificará o conteúdo daquela posição.

Caso exista um inimigo no nó atual:

1. O movimento do herói será interrompido;
2. Os dados do inimigo serão enviados para a Fila Circular de Combate;
3. O sistema iniciará o processamento das ações de batalha.

Quando um inimigo for derrotado:

- itens poderão ser enviados para a Pilha de Inventário;
- cartas poderão ser adicionadas à Lista Duplamente Encadeada de Cartas;
- experiência e atributos do herói serão atualizados.

Ao utilizar uma carta:

- a carta será removida da lista de cartas;
- a matriz do cenário será atualizada;
- modificações poderão afetar diretamente nós da lista circular do mapa.

Dessa forma, todas as estruturas operam em conjunto, permitindo que o funcionamento interno do jogo represente diretamente os conceitos estudados em Estrutura de Dados.

4. Requisitos Funcionais (RF)

4. Requisitos Funcionais (RF)

Os requisitos funcionais descrevem as funcionalidades e comportamentos esperados do sistema do jogo, especificando as ações que o jogador poderá realizar e as respostas do sistema durante a execução.

4.1 Sistema de Cadastro e Login

RF01 – Cadastro de Jogador

O sistema deve permitir que o jogador realize cadastro utilizando nome de usuário, e-mail e senha.

RF02 – Login de Jogador

O sistema deve permitir autenticação do jogador por meio de e-mail e senha.

4.2 Sistema de Personagem

RF03 – Criação de Personagem

O sistema deve permitir ao jogador criar um personagem selecionando nome, aparência e classe inicial.

RF04 – Seleção de Classe

O sistema deve disponibilizar diferentes classes jogáveis, cada uma com atributos e habilidades específicas.

RF06 – Evolução de Personagem

O sistema deve permitir evolução do personagem através de níveis, atributos e habilidades.

RF07 – Sistema de Vida e Energia

O sistema deve controlar pontos de vida, resistência e outros atributos do personagem.

4.3 Sistema de Gameplay

RF08 – Movimentação Automática

O personagem deve percorrer automaticamente o caminho definido no mapa.

RF09 – Geração de Inimigos

O sistema deve gerar inimigos dinamicamente conforme estruturas e cartas posicionadas no mapa.

RF10 – Sistema de Combate

O sistema deve iniciar combate automaticamente quando o personagem encontrar inimigos.

RF11– Cálculo de Dano

O sistema deve calcular dano considerando ataque, defesa, equipamentos e habilidades.

RF12 – Derrota de Inimigos

O sistema deve remover inimigos derrotados e liberar recompensas ao jogador.

RF13 – Sistema de Loot

O sistema deve gerar recompensas aleatórias após batalhas, incluindo equipamentos, recursos e cartas.

RF14– Progressão de Dificuldade

O sistema deve aumentar gradativamente a dificuldade conforme o avanço da partida.

4.4 Sistema de Cartas e Construção do Mapa

RF15 – Coleta de Cartas

O sistema deve fornecer cartas ao jogador durante a exploração e batalhas.

RF16 – Posicionamento de Cartas

O sistema deve permitir posicionar cartas em espaços válidos do mapa.

RF17 – Alteração do Ambiente

O sistema deve modificar o ambiente e os eventos do jogo de acordo com as cartas posicionadas.

RF18 – Geração de Estruturas

O sistema deve criar estruturas, terrenos e construções baseadas nas cartas utilizadas.

RF19 – Interação entre Cartas

O sistema deve reconhecer combinações específicas entre cartas e gerar efeitos especiais.

4.5 Sistema de Inventário e Equipamentos

RF20 – Inventário do Jogador

O sistema deve armazenar itens coletados pelo jogador.

RF21 – Equipamento de Itens

O sistema deve permitir equipar armas, armaduras, acessórios e itens especiais.

RF22 – Comparação de Equipamentos

O sistema deve apresentar comparação entre atributos de equipamentos.

RF23 – Remoção de Itens

O sistema deve permitir descartar ou substituir itens do inventário.

RF24 – Limite de Inventário

O sistema deve controlar quantidade máxima de itens armazenados.

4.6 Sistema de Acampamento

RF25 – Retorno ao Acampamento

O sistema deve permitir retorno ao acampamento para gerenciamento de recursos.

RF26 – Construção de Estruturas

O sistema deve permitir construção e melhoria de estruturas no acampamento.

RF27 – Bônus Permanentes

O sistema deve fornecer melhorias permanentes ao jogador através das construções.

RF28 – Gerenciamento de Recursos

O sistema deve controlar materiais coletados e gastos em construções.

4.7 Sistema de Interface

RF29 – Interface Principal

O sistema deve apresentar menu principal com opções de jogar, configurações e sair.

RF30 – Interface de Gameplay

O sistema deve exibir informações de vida, equipamentos, cartas, mapa e recursos em tempo real.

RF31 – Interface de Inventário

O sistema deve permitir visualização organizada dos itens coletados.

RF32 – Interface de Configurações

O sistema deve permitir alteração de áudio, gráficos e controles.

RF33 – Sistema de Pause

O sistema deve permitir pausar e retomar a partida.

4.8 Sistema de Salvamento

RF34 – Salvamento Automático

O sistema deve salvar automaticamente o progresso durante a partida.

RF35 – Salvamento Manual

O sistema deve permitir ao jogador salvar manualmente o progresso.

RF36 – Carregamento de Partida

O sistema deve permitir carregar partidas salvas anteriormente.

4.9 Sistema de Eventos e Game Over

RF37 – Eventos Aleatórios

O sistema deve gerar eventos aleatórios durante a exploração.

RF38 – Condição de Derrota

O sistema deve encerrar a partida quando os pontos de vida do personagem chegarem a zero.

RF39– Tela de Game Over

O sistema deve exibir estatísticas e resultados após derrota do jogador.

RF40 – Reinício de Partida

O sistema deve permitir iniciar uma nova partida após o encerramento da anterior.

4.10 Sistema de Estruturas de Dados

RF41 – Gerenciamento de Combate com Fila

O sistema deve utilizar estrutura de fila circular para organização das ações de combate.

RF42 – Gerenciamento de Loot com Lista Encadeada

O sistema deve utilizar listas encadeadas para armazenamento e gerenciamento de itens e recompensas.

RF43 – Gerenciamento de Menus com Pilha

O sistema deve utilizar pilha dinâmica para controle de menus e navegação de interfaces.

RF44 – Comunicação entre Estruturas

O sistema deve integrar as estruturas de dados ao funcionamento do gameplay em tempo real.

4.11 Sistema de Áudio e Feedback

RF45 – Reprodução de Música

O sistema deve reproduzir música de fundo durante o jogo.

RF46 – Efeitos Sonoros

O sistema deve reproduzir efeitos sonoros em combates, movimentação e interações.

RF47 – Feedback Visual

O sistema deve apresentar animações e indicadores visuais para ações do jogador.

RF48 – Notificações do Sistema

O sistema deve informar eventos importantes através de mensagens na interface.

5. Casos de Uso (UC)

O passo a passo detalhado de como o jogador interage com os requisitos.

- Exemplo para o UC01 (Posicionar Carta):
 1. O jogador aperta a tecla de inventário.
 2. O sistema exibe as cartas da Fila/Vetor.
 3. O jogador escolhe a carta "Cemitério" e digita as coordenadas do cenário.
 4. O sistema valida se o espaço está livre, altera a célula correspondente e remove a carta da mão.

5. Casos de Uso (CSU) (**pronto**)

5.1 CSU01 – Login de Jogador

Descrição Resumida

Permite ao jogador acessar sua conta.

Descrição Completa

O jogador informa e-mail e senha na interface de login. O sistema verifica as credenciais cadastradas e, caso estejam corretas, libera acesso ao menu principal e aos dados salvos do usuário.

5.3 CSU03 – Criar Personagem

Descrição Resumida

Permite criar um novo personagem jogável.

Descrição Completa

O jogador seleciona nome, aparência e classe inicial do personagem. O sistema cria o personagem com atributos e habilidades compatíveis com a classe escolhida.

5.4 CSU04 – Selecionar Classe

Descrição Resumida

Permite escolher uma classe jogável.

Descrição Completa

Durante a criação do personagem, o jogador escolhe entre diferentes classes disponíveis. O sistema atribui atributos, habilidades e características específicas conforme a classe selecionada.

5.5 CSU05 – Evoluir Personagem

Descrição Resumida

Permite aumentar atributos e habilidades do personagem.

Descrição Completa

Conforme o jogador derrota inimigos e progride na partida, o sistema concede experiência ao personagem. Ao subir de nível, atributos e habilidades são aprimorados automaticamente ou através de escolhas do jogador.

5.6 CSU06 – Gerenciar Vida e Energia

Descrição Resumida

Permite controlar os atributos vitais do personagem.

Descrição Completa

O sistema monitora continuamente pontos de vida, resistência e demais atributos do personagem durante exploração e combate, realizando atualizações em tempo real conforme dano, recuperação ou uso de habilidades.

5.7 CSU07 – Movimentação Automática

Descrição Resumida

Permite ao personagem percorrer automaticamente o mapa.

Descrição Completa

Após o início da partida, o personagem se movimenta automaticamente pelo caminho disponível no mapa. Durante o percurso, eventos, inimigos e recompensas podem ser acionados.

5.8 CSU08 – Gerar Inimigos

Descrição Resumida

Permite a criação dinâmica de inimigos no mapa.

Descrição Completa

O sistema gera inimigos automaticamente de acordo com estruturas, cartas e progresso do jogador, aumentando a variedade e dificuldade dos desafios durante a exploração.

5.9 CSU09 – Iniciar Combate

Descrição Resumida

Permite iniciar batalhas automaticamente.

Descrição Completa

Quando o personagem encontra inimigos no caminho, o sistema inicia automaticamente o combate, ativando cálculos de ataque, defesa, habilidades e efeitos especiais.

5.10 CSU10 – Calcular Dano

Descrição Resumida

Permite calcular o dano causado durante batalhas.

Descrição Completa

O sistema calcula o dano considerando atributos do personagem, equipamentos equipados, habilidades ativas e defesa dos inimigos, atualizando os pontos de vida em tempo real.

5.11 CSU11 – Derrotar Inimigos

Descrição Resumida

Permite concluir batalhas contra inimigos.

Descrição Completa

Quando os pontos de vida do inimigo chegam a zero, o sistema remove o inimigo do cenário e libera recompensas para o jogador, incluindo experiência, recursos e itens.

5.12 CSU12 – Coletar Loot

Descrição Resumida

Permite obter recompensas após combates.

Descrição Completa

Após derrotar inimigos, o sistema gera recompensas aleatórias como equipamentos, recursos e cartas, armazenando automaticamente os itens obtidos no inventário do jogador.

5.13 CSU13 – Aumentar Dificuldade

Descrição Resumida

Permite evolução gradual da dificuldade do jogo.

Descrição Completa

Conforme o jogador avança na exploração, o sistema aumenta gradativamente atributos dos inimigos, frequência de eventos e complexidade dos desafios presentes no mapa.

5.14 CSU14 – Coletar Cartas

Descrição Resumida

Permite adquirir cartas durante a partida.

Descrição Completa

O sistema fornece cartas ao jogador após batalhas, eventos ou exploração. Essas cartas podem ser utilizadas posteriormente para alterar o mapa e gerar estruturas.

5.15 CSU15 – Posicionar Cartas

Descrição Resumida

Permite utilizar cartas no mapa.

Descrição Completa

O jogador seleciona cartas disponíveis e posiciona em espaços válidos do mapa. O sistema valida a posição escolhida e aplica os efeitos correspondentes.

5.16 CSU16 – Alterar Ambiente

Descrição Resumida

Permite modificar o cenário do jogo.

Descrição Completa

Após o posicionamento das cartas, o sistema altera o ambiente, adicionando efeitos, modificadores de terreno, bônus ou novos desafios no mapa.

5.17 CSU17 – Gerar Estruturas

Descrição Resumida

Permite criar estruturas e terrenos especiais.

Descrição Completa

O sistema gera automaticamente construções, terrenos e estruturas conforme as cartas utilizadas pelo jogador durante a gameplay.

5.18 CSU18 – Interagir Cartas

Descrição Resumida

Permite ativar combinações especiais entre cartas.

Descrição Completa

O sistema identifica combinações específicas de cartas posicionadas próximas entre si e ativa efeitos especiais, bônus ou eventos únicos.

5.19 CSU19 – Gerenciar Inventário

Descrição Resumida

Permite armazenar e visualizar itens coletados.

Descrição Completa

O jogador acessa o inventário para visualizar equipamentos, recursos e consumíveis. O sistema organiza os itens armazenados e atualiza o conteúdo em tempo real.

5.20 CSU20 – Equipar Itens

Descrição Resumida

Permite utilizar equipamentos no personagem.

Descrição Completa

O jogador seleciona armas, armaduras ou acessórios disponíveis no inventário. O sistema equipa os itens escolhidos e atualiza os atributos do personagem.

5.21 CSU21 – Comparar Equipamentos

Descrição Resumida

Permite comparar atributos de itens.

Descrição Completa

Ao selecionar um equipamento, o sistema apresenta comparação entre atributos do item atual e do novo equipamento, auxiliando o jogador na escolha.

5.22 CSU22 – Remover Itens

Descrição Resumida

Permite descartar itens do inventário.

Descrição Completa

O jogador pode remover ou substituir itens armazenados. O sistema atualiza imediatamente o inventário e libera espaço para novos objetos.

5.23 CSU23 – Controlar Limite de Inventário

Descrição Resumida

Permite controlar capacidade máxima de itens.

Descrição Completa

O sistema monitora continuamente a quantidade de itens armazenados no inventário e impede exceder o limite definido.

5.24 CSU24 – Retornar ao Acampamento

Descrição Resumida

Permite acessar a área principal de gerenciamento.

Descrição Completa

O jogador pode retornar ao acampamento para administrar recursos, construções e melhorias permanentes desbloqueadas durante a partida..

5.28 CSU28 – Acessar Interface Principal

Descrição Resumida

Permite acessar o menu principal do jogo.

Descrição Completa

O sistema apresenta opções principais como iniciar partida, carregar jogo, configurações e sair, permitindo navegação entre funcionalidades do sistema.

5.29 CSU29 – Exibir Interface de Gameplay

Descrição Resumida

Permite visualizar informações da partida.

Descrição Completa

Durante a gameplay, o sistema exibe informações em tempo real como vida, equipamentos, cartas, recursos, minimapa e progresso da partida.

5.30 CSU30 – Exibir Interface de Inventário

Descrição Resumida

Permite visualizar itens organizados.

Descrição Completa

O sistema apresenta uma interface organizada contendo todos os itens coletados, categorias de equipamentos e espaços disponíveis no inventário.

5.31 CSU31 – Configurar Jogo

Descrição Resumida

Permite alterar configurações do sistema.

Descrição Completa

O jogador pode modificar configurações de áudio, vídeo e controles. O sistema salva as preferências e aplica as alterações selecionadas.

5.32 CSU32 – Pausar Partida

Descrição Resumida

Permite interromper temporariamente a gameplay.

Descrição Completa

O jogador pode pausar a partida a qualquer momento. O sistema interrompe movimentação, combate e eventos até que a partida seja retomada.

5.33 CSU33 – Salvar Progresso

Descrição Resumida

Permite salvar dados da partida.

Descrição Completa

O sistema salva automaticamente ou manualmente o progresso do jogador, incluindo inventário, equipamentos, estruturas e evolução do personagem.

5.34 CSU34 – Carregar Partida

Descrição Resumida

Permite continuar partidas salvas.

Descrição Completa

O jogador seleciona um salvamento disponível e o sistema restaura informações anteriormente armazenadas, retomando a gameplay do ponto salvo.

5.35 CSU35 – Gerar Eventos Aleatórios

Descrição Resumida

Permite ocorrência de eventos dinâmicos.

Descrição Completa

Durante a exploração, o sistema gera eventos aleatórios capazes de alterar desafios, recompensas ou condições da partida.

5.36 CSU36 – Encerrar Partida

Descrição Resumida

Permite finalizar a partida após derrota.

Descrição Completa

Quando os pontos de vida do personagem chegam a zero, o sistema encerra a partida atual, exibindo tela de derrota e estatísticas obtidas durante a sessão.

5.37 CSU37 – Reiniciar Partida

Descrição Resumida

Permite iniciar uma nova tentativa.

Descrição Completa

Após o encerramento da partida, o jogador pode iniciar uma nova jornada. O sistema reinicia a gameplay mantendo progressões permanentes desbloqueadas.

5.38 CSU38 – Gerenciar Estruturas de Dados

Descrição Resumida

Permite integração das estruturas de dados ao jogo.

Descrição Completa

O sistema utiliza fila circular para gerenciamento de combate, listas encadeadas para controle de loot e pilhas dinâmicas para navegação entre menus, garantindo funcionamento integrado da gameplay em tempo real.

5.39 CSU39 – Reproduzir Áudio

Descrição Resumida

Permite execução de músicas e efeitos sonoros.

Descrição Completa

O sistema reproduz músicas de fundo e efeitos sonoros durante movimentação, combates e interações, aumentando a imersão do jogador.

5.40 CSU40 – Exibir Feedback Visual

Descrição Resumida

Permite apresentar respostas visuais ao jogador.

Descrição Completa

O sistema exibe animações, indicadores, notificações e mensagens informativas para representar ações, eventos e mudanças importantes durante a gameplay.

Diagrama de caso de uso:

código: @startuml

left to right direction

skinparam packageStyle rectangle

skinparam usecase {

 BackgroundColor #E8E8E8

 BorderColor Black

 ArrowColor Black

}

actor "Jogador" as Player

rectangle "Sistema do Jogo" {

 usecase "Cadastro de\nJogador" as UC1

```
usecase "Login de\nJogador" as UC2
usecase "Criar\nPersonagem" as UC3
usecase "Selecionar\nClasse" as UC4
usecase "Explorar\nMapa" as UC5
usecase "Entrar em\nCombate" as UC6
usecase "Calcular\nDano" as UC7
usecase "Coletar\nLoot" as UC8
usecase "Gerenciar\nInventário" as UC9
usecase "Equipar\nItens" as UC10
usecase "Utilizar\nCartas" as UC11
usecase "Construir\nEstruturas" as UC12
usecase "Gerenciar\nAcampamento" as UC13
usecase "Salvar\nPartida" as UC14
usecase "Carregar\nPartida" as UC15
usecase "Configurar\nJogo" as UC16
usecase "Pausar\nPartida" as UC17
usecase "Reiniciar\nPartida" as UC18
}
```

Player --> UC1

Player --> UC2

Player --> UC3

Player --> UC5

Player --> UC9

Player --> UC11

Player --> UC13

Player --> UC14

Player --> UC15

Player --> UC16

Player --> UC17

Player --> UC18

UC3 .> UC4 : <<include>>

UC5 .> UC6 : <<extend>>

UC5 .> UC8 : <<extend>>

UC6 .> UC7 : <<include>>

UC6 .> UC8 : <<include>>

UC9 .> UC10 : <<include>>

UC11 .> UC12 : <<include>>

@enduml

6. Requisitos Não Funcionais (RNF) **(revisar a partir daqui)**

5.1 RNF01 – Requisitos de Informação (RIN)

RNF01.1 – Armazenamento de Dados

O sistema deve armazenar corretamente informações relacionadas ao progresso do jogador, incluindo nível, atributos, equipamentos, inventário, recursos e evolução da campanha.

RNF01.2 – Exibição de Informações

As informações exibidas durante a gameplay devem ser organizadas, claras e de fácil compreensão, permitindo rápida interpretação pelo jogador.

RNF01.3 – Persistência de Dados

O progresso do jogador deve ser salvo automaticamente em eventos importantes, como retorno ao acampamento, derrota de chefes e encerramento da sessão.

RNF01.4 – Integridade das Informações

Os dados armazenados não devem sofrer alterações inesperadas durante falhas ou encerramentos inadequados do sistema.

5.2 RNF02 – Requisitos de Segurança da Informação (RSI)

RNF02.1 – Proteção de Salvamentos

Os arquivos de salvamento devem ser protegidos contra corrupção e perda de dados.

RNF02.2 – Recuperação de Dados

O sistema deve possuir mecanismos de recuperação automática de dados em caso de falhas inesperadas.

RNF02.3 – Integridade do Sistema

As informações internas do jogo não devem ser alteradas indevidamente durante a execução.

RNF02.4 – Segurança de Arquivos

Os dados do jogador devem ser armazenados localmente de maneira segura e organizada.

5.3 RNF03 – Requisitos de Usabilidade e Interface de Texto/Gráfica (RUSA)

RNF03.1 – Navegação

Os menus devem possuir navegação intuitiva, permitindo acesso rápido às funcionalidades principais do sistema.

RNF03.2 – Organização Visual

Os elementos gráficos da interface devem seguir padronização visual consistente, facilitando a identificação de botões, menus e informações importantes.

RNF03.3 – Feedback Visual

Botões, ações e interações do jogador devem apresentar respostas visuais imediatas, indicando sucesso, erro ou seleção realizada.

RNF03.4 – Tutorial

O jogo deve possuir tutorial inicial explicando mecânicas básicas, combate, movimentação e utilização das estruturas principais.

RNF03.5 – Clareza da Interface

Os textos, ícones e indicadores devem possuir tamanho adequado e boa legibilidade durante toda a gameplay.

5.4 RNF04 – Requisitos de Acessibilidade (RACE)

RNF04.1 – Ajuste de Áudio

O sistema deve permitir controle individual de volume para músicas, efeitos sonoros e sons da interface.

RNF04.2 – Ajuste de Resolução

O jogador deve poder alterar resolução e modo de exibição da tela.

RNF04.3 – Contraste Visual

O jogo deve possuir opções de contraste e brilho para melhorar visualização dos elementos gráficos.

RNF04.4 – Legibilidade

Os textos exibidos devem possuir tamanho e contraste adequados para leitura confortável.

5.5 RNF05 – Requisitos de Confiabilidade (RCON)

RNF05.1 – Estabilidade

O sistema deve minimizar travamentos e falhas durante longos períodos de execução.

RNF05.2 – Continuidade da Gameplay

O jogo deve continuar funcionando corretamente mesmo após múltiplas interações consecutivas do jogador.

RNF05.3 – Confiabilidade do Salvamento

Os dados salvos devem permanecer íntegros após encerramentos normais ou inesperados do jogo.

RNF05.4 – Tratamento de Erros

O sistema deve tratar erros internos sem comprometer a experiência do jogador.

5.6 RNF06 – Requisitos de Desempenho e Eficiência (RDES)

RNF06.1 – Taxa de Quadros

O jogo deve manter taxa mínima de 60 FPS em computadores com configuração intermediária.

RNF06.2 – Tempo de Carregamento

As telas do sistema devem carregar em até 2 segundos.

RNF06.3 – Resposta de Comandos

As ações realizadas pelo jogador devem possuir resposta imediata durante menus e gameplay.

RNF06.4 – Consumo de Recursos

O sistema deve otimizar o uso de memória RAM e processamento durante longas sessões.

RNF06.5 – Complexidade Algorítmica

As operações de inserção e remoção em estruturas de dados devem priorizar eficiência computacional, buscando operações $O(1)$ sempre que possível.

5.7 RNF07 – Requisitos de Disponibilidade (RDIS)

RNF07.1 – Funcionamento Offline

O jogo deve funcionar completamente sem conexão com internet.

RNF07.2 – Disponibilidade do Sistema

As funcionalidades principais devem permanecer acessíveis durante toda a execução do sistema.

RNF07.3 – Continuidade de Sessão

O jogador deve conseguir continuar a gameplay sem interrupções causadas por dependências externas.

5.8 RNF08 – Requisitos de Segurança de Acesso (RSEG)

RNF08.1 – Controle de Acesso

O acesso aos arquivos internos do jogo deve ocorrer apenas pelo próprio sistema.

RNF08.2 – Proteção de Dados

O sistema deve evitar modificações indevidas nos dados salvos do jogador.

RNF08.3 – Validação de Informações

.

As informações carregadas pelo sistema devem ser verificadas antes da execução

5.9 RNF09 – Requisitos de Manutenibilidade e Gerenciamento de Memória (RMAN)

RNF09.1 – Modularidade

O código-fonte deve ser organizado em módulos independentes para facilitar manutenção e expansão.

RNF09.2 – Documentação Interna

As funções e estruturas utilizadas devem possuir comentários explicativos.

RNF09.3 – Reutilização de Código

Os componentes do sistema devem permitir reutilização em diferentes partes do projeto.

RNF09.4 – Gerenciamento de Memória

As estruturas dinâmicas devem ser desalocadas corretamente ao encerrar o jogo.

RNF09.5 – Prevenção de Vazamento de Memória

O sistema deve evitar memory leaks durante execução prolongada.

5.10 RNF10 – Requisitos de Portabilidade (RPORT)

RNF10.1 – Compatibilidade de Sistemas

O jogo deve funcionar corretamente em Windows e Linux.

RNF10.2 – Adaptação Futura

A arquitetura do projeto deve permitir futura adaptação para outras plataformas.

RNF10.3 – Independência de Plataforma

As bibliotecas utilizadas devem possuir compatibilidade multiplataforma.

5.11 RNF11 – Requisitos de Documentação (RDOC)

RNF11.1 – Documentação Técnica

O projeto deve possuir documentação das estruturas de dados e algoritmos utilizados.

RNF11.2 – Organização Acadêmica

A documentação deve seguir padronização acadêmica adequada.

RNF11.3 – Clareza de Informações

As informações documentadas devem ser claras e organizadas.

5.12 RNF12 – Requisitos de Interface Homem-Computador (RHIC)

RNF12.1 – Feedback Sonoro

As ações importantes devem possuir resposta sonora correspondente.

RNF12.2 – Interatividade

O sistema deve responder rapidamente às interações do jogador.

RNF12.3 – Comunicação Visual

Eventos importantes devem ser apresentados visualmente ao usuário.

5.13 RNF13 – Requisitos de Interface Externa (RIEX)

RNF13.1 – Compatibilidade de Dispositivos

O sistema deve suportar teclado e mouse como dispositivos principais.

RNF13.2 – Manipulação de Arquivos

O jogo deve permitir leitura e gravação correta de arquivos de salvamento.

RNF13.3 – Comunicação Externa

As interfaces externas devem funcionar sem comprometer desempenho do sistema.

5.14 RNF14 – Requisitos de Plataforma de Hardware (RPHW)

RNF14.1 – Configuração Mínima

O jogo deve executar em computadores com configuração intermediária.

RNF14.2 – Estabilidade de Hardware

O consumo de processamento e memória deve permanecer estável.

RNF14.3 – Compatibilidade de Resolução

O sistema deve suportar resoluções padrão desktop.

5.15 RNF15 – Requisitos de Plataforma de Software (RPSW)

RNF15.1 – Compatibilidade de Bibliotecas

O sistema deve utilizar bibliotecas estáveis e compatíveis com múltiplos sistemas operacionais.

RNF15.2 – Atualização de Software

O projeto deve manter compatibilidade com versões atuais dos sistemas suportados.

RNF15.3 – Estabilidade do Ambiente

As dependências do sistema devem garantir execução contínua e segura do jogo.

7. Identificação por TAG dos Artefatos

- 7.1 Padrão de TAGs do Projeto
- 7.2 Convenções de Nomeação de Variáveis e Funções

8. Rastreabilidade dos Artefatos

- 8.1 Rastreabilidade Vertical (Requisitos $\rightarrow$ Código)
- 8.2 Rastreabilidade Horizontal (Casos de Uso $\rightarrow$ Casos de Teste)

11. Tabela de Casos de Teste (CT)

CT01 – Cadastro de Jogador

Objetivo

Validar o cadastro de novos jogadores no sistema.

Descrição Resumida

Verificar se o sistema permite o cadastro de usuários utilizando nome, e-mail e senha.

Descrição Completa

O sistema deve permitir que o usuário realize cadastro informando dados válidos. Após a confirmação, as informações devem ser armazenadas corretamente para permitir acesso futuro ao jogo.

Pré-Condições

- O jogador não deve possuir cadastro prévio.
- O sistema deve estar online.

Passos de Execução

1. Acessar a tela de cadastro.
2. Informar nome de usuário.
3. Informar e-mail válido.
4. Informar senha.
5. Confirmar cadastro.

Resultado Esperado

O cadastro deve ser realizado com sucesso e o jogador deve conseguir acessar o sistema.

CT02 – Login de Jogador

Objetivo

Validar autenticação de jogadores cadastrados.

Descrição Resumida

Verificar se o sistema realiza login utilizando e-mail e senha.

Descrição Completa

O sistema deve validar as credenciais informadas pelo jogador. Caso estejam corretas, o acesso ao menu principal deve ser liberado.

Pré-Condições

- O jogador deve possuir cadastro ativo.

Passos de Execução

1. Acessar a tela de login.
2. Informar e-mail cadastrado.
3. Informar senha correta.
4. Confirmar login.

Resultado Esperado

O sistema deve autenticar o jogador e liberar acesso ao jogo.

CT03 – Criação de Personagem

Objetivo

Validar criação de personagens.

Descrição Resumida

Verificar se o jogador consegue criar um personagem.

Descrição Completa

O sistema deve permitir seleção de nome, classe e aparência do personagem. Após confirmação, o personagem deve ser salvo corretamente.

Pré-Condições

- O jogador deve estar autenticado.

Passos de Execução

1. Acessar menu de criação.
2. Escolher classe.
3. Escolher aparência.
4. Definir nome.
5. Confirmar criação.

Resultado Esperado

O personagem deve ser criado e salvo corretamente.

CT04 – Seleção de Personagem

Objetivo

Validar seleção de personagens existentes.

Descrição Resumida

Verificar se o jogador consegue selecionar personagens salvos.

Descrição Completa

O sistema deve listar os personagens disponíveis e permitir que o jogador escolha um para iniciar a partida.

Pré-Condições

- Deve existir pelo menos um personagem salvo.

Passos de Execução

1. Acessar lista de personagens.
2. Selecionar personagem.
3. Confirmar seleção.

Resultado Esperado

O personagem selecionado deve iniciar corretamente no jogo.

CT05 – Movimentação Automática

Objetivo

Validar movimentação automática do personagem.

Descrição Resumida

Verificar se o personagem percorre automaticamente o caminho do mapa.

Descrição Completa

O sistema deve movimentar automaticamente o personagem durante a exploração do mapa, carregando eventos e inimigos corretamente.

Pré-Condições

- Uma partida deve estar iniciada.

Passos de Execução

1. Iniciar partida.
2. Observar movimentação do personagem.
3. Verificar continuidade do percurso.

Resultado Esperado

O personagem deve percorrer o caminho sem interrupções indevidas.

CT06 – Sistema de Combate

Objetivo

Validar funcionamento do combate automático.

Descrição Resumida

Verificar combate entre personagem e inimigos.

Descrição Completa

Ao encontrar inimigos, o sistema deve iniciar combate automaticamente, aplicando atributos, dano e defesa corretamente.

Pré-Condições

- Deve existir inimigo no mapa.

Passos de Execução

1. Iniciar exploração.
2. Encontrar inimigo.
3. Aguardar combate.
4. Verificar troca de ataques.

Resultado Esperado

O combate deve ocorrer corretamente até a derrota de um dos lados.

CT07 – Coleta de Loot

Objetivo

Validar obtenção de recompensas.

Descrição Resumida

Verificar coleta de itens após batalhas.

Descrição Completa

Após derrotar inimigos, o sistema deve gerar itens e recursos automaticamente para o inventário do jogador.

Pré-Condições

- O jogador deve vencer um combate.

Passos de Execução

1. Derrotar inimigo.
2. Verificar itens recebidos.
3. Abrir inventário.

Resultado Esperado

Os itens obtidos devem aparecer corretamente no inventário.

CT08 – Gerenciamento de Inventário

Objetivo

Validar manipulação de itens.

Descrição Resumida

Verificar organização e gerenciamento do inventário.

Descrição Completa

O sistema deve permitir visualizar, equipar e remover itens do inventário sem inconsistências.

Pré-Condições

- O jogador deve possuir itens.

Passos de Execução

1. Abrir inventário.
2. Selecionar item.
3. Equipar ou remover item.
4. Confirmar alteração.

Resultado Esperado

O inventário deve atualizar corretamente as informações.

CT09 – Equipamento de Itens

Objetivo

Validar sistema de equipamentos.

Descrição Resumida

Verificar equipagem de armas e armaduras.

Descrição Completa

O sistema deve permitir substituir equipamentos e atualizar atributos do personagem automaticamente.

Pré-Condições

- O jogador deve possuir equipamentos.

Passos de Execução

1. Abrir inventário.
2. Selecionar equipamento.
3. Equipar item.
4. Verificar atributos.

Resultado Esperado

O item deve ser equipado corretamente e os atributos atualizados.

CT10 – Construção do Mapa

Objetivo

Validar posicionamento de cartas no mapa.

Descrição Resumida

Verificar construção e modificação do mapa.

Descrição Completa

O sistema deve permitir posicionar cartas no mapa, alterando terrenos, estruturas e efeitos.

Pré-Condições

- O jogador deve possuir cartas disponíveis.

Passos de Execução

1. Selecionar carta.
2. Escolher posição no mapa.
3. Confirmar posicionamento.

Resultado Esperado

A carta deve modificar corretamente o mapa.

CT11 – Construção do Acampamento

Objetivo

Validar melhorias do acampamento.

Descrição Resumida

Verificar criação de estruturas no acampamento.

Descrição Completa

O sistema deve permitir utilizar recursos para construir melhorias permanentes.

Pré-Condições

- O jogador deve possuir recursos suficientes.

Passos de Execução

1. Acessar acampamento.
2. Selecionar construção.
3. Confirmar utilização de recursos.

Resultado Esperado

A construção deve ser concluída corretamente.

CT12 – Progressão de Personagem

Objetivo

Validar evolução do personagem.

Descrição Resumida

Verificar ganho de experiência e subida de nível.

Descrição Completa

O personagem deve ganhar experiência ao derrotar inimigos e evoluir ao atingir a quantidade necessária.

Pré-Condições

- O personagem deve participar de combates.

Passos de Execução

1. Derrotar inimigos.
2. Verificar ganho de experiência.
3. Acompanhar subida de nível.

Resultado Esperado

O personagem deve evoluir corretamente.

CT13 – Salvamento de Progresso

Objetivo

Validar salvamento de dados.

Descrição Resumida

Verificar salvamento do progresso do jogo.

Descrição Completa

O sistema deve salvar inventário, personagem, equipamentos e progresso da partida.

Pré-Condições

- Deve existir progresso no jogo.

Passos de Execução

1. Jogar normalmente.
2. Salvar partida.
3. Encerrar jogo.
4. Reabrir jogo.
5. Carregar save.

Resultado Esperado

Os dados devem ser recuperados corretamente.

CT14 – Configurações do Sistema

Objetivo

Validar configurações do jogo.

Descrição Resumida

Verificar alteração das configurações.

Descrição Completa

O jogador deve conseguir alterar áudio, gráficos e controles.

Pré-Condições

- O jogo deve estar em execução.

Passos de Execução

1. Abrir configurações.
2. Alterar opções.
3. Salvar alterações.
4. Reiniciar jogo.

Resultado Esperado

As configurações devem permanecer salvas.

CT15 – Tela de Game Over

Objetivo

Validar encerramento da partida.

Descrição Resumida

Verificar exibição da tela de derrota.

Descrição Completa

Quando a vida do personagem chegar a zero, o sistema deve finalizar a partida e apresentar informações da sessão.

Pré-Condições

- O personagem deve estar em combate.

Passos de Execução

1. Receber dano até zerar vida.
2. Aguardar encerramento.

Resultado Esperado

A tela de Game Over deve ser exibida corretamente.

CT16 – Sistema de Pausa

Objetivo

Validar pausa da partida.

Descrição Resumida

Verificar interrupção temporária do jogo.

Descrição Completa

O sistema deve interromper movimentação, combate e eventos enquanto o jogo estiver pausado.

Pré-Condições

- A partida deve estar em andamento.

Passos de Execução

1. Iniciar partida.
2. Acionar pausa.
3. Verificar interrupção.
4. Retornar ao jogo.

Resultado Esperado

O jogo deve pausar e retornar corretamente.

CT17 – Interface do Usuário

Objetivo

Validar elementos visuais do sistema.

Descrição Resumida

Verificar funcionamento da interface gráfica.

Descrição Completa

O sistema deve apresentar corretamente menus, barras, inventário, cartas e demais elementos gráficos.

Pré-Condições

- O jogo deve estar iniciado.

Passos de Execução

1. Navegar pelos menus.
2. Abrir inventário.
3. Verificar elementos gráficos.

Resultado Esperado

A interface deve funcionar sem erros visuais.

CT18 – Desempenho do Sistema

Objetivo

Validar estabilidade do jogo.

Descrição Resumida

Verificar desempenho durante execução prolongada.

Descrição Completa

O sistema deve manter estabilidade mesmo com múltiplos inimigos e efeitos ativos.

Pré-Condições

- O jogo deve estar em execução.

Passos de Execução

1. Jogar por período prolongado.
2. Ativar múltiplos elementos.
3. Monitorar desempenho.

Resultado Esperado

O jogo deve funcionar sem travamentos críticos.

CT19 – Compatibilidade do Sistema

Objetivo

Validar compatibilidade do jogo.

Descrição Resumida

Verificar funcionamento em diferentes ambientes.

Descrição Completa

O sistema deve funcionar corretamente em resoluções e sistemas operacionais suportados.

Pré-Condições

- O jogo deve estar instalado.

Passos de Execução

1. Executar jogo em diferentes resoluções.
2. Testar em diferentes ambientes.
3. Verificar interface.

Resultado Esperado

O jogo deve funcionar corretamente nos ambientes suportados.

CT20 – Encerramento do Sistema

Objetivo

Validar encerramento correto do jogo.

Descrição Resumida

Verificar saída segura do sistema.

Descrição Completa

O sistema deve permitir encerramento correto do jogo sem perda de dados.

Pré-Condições

- O jogo deve estar em execução.

Passos de Execução

1. Abrir menu principal.
2. Selecionar opção sair.
3. Confirmar encerramento.

Resultado Esperado

O jogo deve ser encerrado corretamente sem apresentar erros.

12. Regras de Negócio (RN)

RN01 – Progressão por Expedição

Descrição Resumida:

O progresso do jogador acontece por meio de expedições realizadas no mapa.

Descrição Completa:

O sistema deve permitir que o jogador evolua seu personagem ao participar de expedições, enfrentando inimigos, coletando recursos e sobrevivendo ao maior número possível de ciclos. Ao finalizar uma expedição, os recursos e experiências adquiridos devem ser utilizados para fortalecer o acampamento e desbloquear melhorias permanentes.

RN02 – Derrota e Perda Parcial de Recursos

Descrição Resumida:

Ao ser derrotado, o jogador perde parte dos recursos coletados.

Descrição Completa:

O sistema deve aplicar penalidades em caso de derrota durante uma expedição. Quando o personagem morrer antes de retornar ao acampamento, parte dos recursos coletados deverá ser perdida automaticamente, mantendo apenas uma porcentagem previamente definida pelo sistema como forma de equilíbrio e progressão.

RN03 – Retorno Seguro ao Acampamento

Descrição Resumida:

O jogador pode retornar ao acampamento para salvar recursos.

Descrição Completa:

O sistema deve permitir que o jogador encerre voluntariamente a expedição ao retornar ao acampamento. Nesse caso, todos os recursos obtidos durante a jornada deverão ser preservados e adicionados ao progresso permanente do jogador.

RN04 – Equipamentos Substituem Itens Inferiores

Descrição Resumida:

Equipamentos melhores substituem os atuais do personagem.

Descrição Completa:

O sistema deve permitir que equipamentos coletados sejam comparados

automaticamente ou manualmente com os itens já equipados pelo jogador. Caso o novo equipamento possua atributos superiores, ele poderá substituir o item anterior para melhorar o desempenho do personagem em combate.

RN05 – Posicionamento Estratégico de Cartas

Descrição Resumida:

As cartas devem influenciar diretamente o mapa e os inimigos.

Descrição Completa:

O sistema deve aplicar efeitos específicos ao posicionamento de cartas no mapa. Cada carta utilizada deverá alterar o ambiente, gerar recursos, modificar atributos de terreno ou influenciar o surgimento e fortalecimento de inimigos próximos.

RN06 – Ciclos Aumentam a Dificuldade

Descrição Resumida:

A dificuldade do jogo aumenta conforme os ciclos avançam.

Descrição Completa:

O sistema deve aumentar gradualmente o nível de dificuldade da expedição a cada ciclo concluído pelo jogador. Os inimigos deverão possuir atributos mais elevados, surgirem em maior quantidade ou apresentar comportamentos mais agressivos conforme o avanço da partida.

RN07 – Construções Desbloqueiam Recursos

Descrição Resumida:

Construções do acampamento liberam novas funcionalidades.

Descrição Completa:

O sistema deve permitir que o jogador utilize recursos coletados para construir estruturas no acampamento. Cada construção deverá desbloquear funcionalidades específicas, como novos personagens, melhorias passivas, equipamentos ou mecânicas adicionais de jogo.

RN08 – Inventário Limitado

Descrição Resumida:

O inventário do jogador possui limite de capacidade.

Descrição Completa:

O sistema deve limitar a quantidade de itens que podem ser carregados durante a expedição. Quando o limite for atingido, o jogador deverá substituir, descartar ou converter itens automaticamente, conforme as regras definidas pelo sistema.

RN09 – Combate Automático

Descrição Resumida:

Os combates ocorrem automaticamente ao encontrar inimigos.

Descrição Completa:

O sistema deve iniciar batalhas automaticamente quando o personagem entrar em contato com inimigos no mapa. Durante o combate, ataques, defesa e habilidades deverão ser executados de acordo com os atributos dos personagens e equipamentos ativos.

RN10 – Classes Possuem Características Exclusivas

Descrição Resumida:

Cada classe deve possuir habilidades e atributos próprios.

Descrição Completa:

O sistema deve diferenciar as classes jogáveis por meio de atributos, equipamentos compatíveis e habilidades exclusivas. Cada personagem deverá oferecer estilos de jogo distintos, incentivando estratégias variadas durante as expedições.

RN11 – Recursos São Necessários para Evolução

Descrição Resumida:

O progresso depende da coleta de recursos durante as expedições.

Descrição Completa:

O sistema deve exigir recursos específicos para desbloqueio de melhorias, construções e evolução geral do jogador. Esses recursos deverão ser obtidos principalmente por meio de combates, exploração do mapa e utilização estratégica de cartas.

RN12 – Inimigos Surgem Conforme o Ambiente

Descrição Resumida:

O tipo de inimigo depende das cartas e estruturas do mapa.

Descrição Completa:

O sistema deve relacionar o surgimento de inimigos aos elementos posicionados pelo jogador no mapa. Determinadas cartas ou combinações de estruturas deverão influenciar diretamente os tipos de criaturas que aparecerão durante a expedição.

RN13 – Limite de Vida do Personagem

Descrição Resumida:

O personagem possui quantidade limitada de vida durante a expedição.

Descrição Completa:

O sistema deve controlar a vida do personagem durante toda a expedição. Quando os pontos de vida chegarem a zero, a expedição deverá ser encerrada automaticamente e as penalidades de derrota aplicadas.

RN14 – Uso Estratégico de Recursos

Descrição Resumida:

Os recursos devem ser administrados estrategicamente.

Descrição Completa:

O sistema deve exigir que o jogador utilize recursos de forma estratégica, já que construções, melhorias e evolução dependem da quantidade acumulada durante as expedições.

RN15 – Chefes Possuem Condições de Aparição

Descrição Resumida:

Chefes aparecem após requisitos específicos serem atendidos.

Descrição Completa:

O sistema deve liberar batalhas contra chefes apenas quando determinadas

condições forem alcançadas, como quantidade de ciclos, expansão do mapa ou progresso do jogador.

RN16 – Habilidades Dependem da Classe

Descrição Resumida:

As habilidades disponíveis variam conforme a classe escolhida.

Descrição Completa:

O sistema deve restringir habilidades especiais de acordo com a classe do personagem, garantindo diferenciação entre estilos de combate e estratégias de progressão.

RN17 – Progressão Permanente do Acampamento

Descrição Resumida:

As melhorias do acampamento permanecem entre partidas.

Descrição Completa:

O sistema deve manter permanentemente as construções e melhorias realizadas no acampamento, mesmo após derrotas ou encerramento das expedições.

RN18 – Loot Gerado Aleatoriamente

Descrição Resumida:

Os itens obtidos devem ser gerados aleatoriamente.

Descrição Completa:

O sistema deve gerar equipamentos e recompensas de maneira aleatória, considerando raridade, dificuldade da expedição e progresso atual do jogador.

RN19 – Cartas Possuem Regras de Posicionamento

Descrição Resumida:

Algumas cartas só podem ser usadas em locais específicos.

Descrição Completa:

O sistema deve validar o posicionamento das cartas no mapa, permitindo sua utilização apenas em áreas compatíveis com suas funções e efeitos.

RN20 – Pausa Durante a Expedição

Descrição Resumida:

O jogador pode pausar a partida a qualquer momento.

Descrição Completa:

O sistema deve permitir a interrupção temporária da expedição sem perda imediata de progresso, possibilitando ao jogador continuar posteriormente do mesmo ponto salvo.

RN21 – Eventos Aleatórios no Mapa

Descrição Resumida:

Eventos aleatórios podem ocorrer durante a exploração.

Descrição Completa:

O sistema deve gerar eventos aleatórios durante a expedição, podendo oferecer recompensas, desafios extras ou modificadores temporários que alterem a jogabilidade.

RN22 – Limite de Equipamentos Ativos

Descrição Resumida:

O personagem possui quantidade limitada de equipamentos equipados.

Descrição Completa:

O sistema deve restringir o número de equipamentos ativos simultaneamente conforme os slots disponíveis da classe escolhida.

RN23 – Evolução Baseada em Experiência

Descrição Resumida:

O personagem evolui ao acumular experiência.

Descrição Completa:

O sistema deve conceder experiência ao derrotar inimigos e concluir eventos, permitindo aumento de nível e melhoria de atributos do personagem.

RN24 – Salvamento Automático de Progresso

Descrição Resumida:

O progresso do jogador deve ser salvo automaticamente.

Descrição Completa:

O sistema deve registrar automaticamente informações importantes da partida e do acampamento para evitar perda de progresso em caso de fechamento inesperado do jogo.

RN25 – Condições de Vitória da Expedição

Descrição Resumida:

A expedição pode ser concluída ao cumprir objetivos específicos.

Descrição Completa:

O sistema deve considerar a expedição concluída quando o jogador atingir objetivos previamente definidos, como derrotar um chefe, sobreviver por determinado número de ciclos ou completar desafios específicos.

13. Tabelas e Matriz de Rastreabilidade

- Tabelas simples resumindo os IDs criados. A Matriz (13) é uma tabela que cruza dados: mostra qual **RF** gerou qual **UC**, que por sua vez gerou qual **CT** (Caso de Teste) no código.

14. Especificações Técnicas (O rascunho do código)

14.1 Definição das Structs: Escrever como serão os seus nós. Exemplo em C:

C

```
typedef struct CaminhoNode {
```

```
    int tipo_terreno;
```

```
    struct CaminhoNode* proximo;
```

```
    struct CaminhoNode* anterior;
```

```
} CaminhoNode;
```

- Use o código com cuidado.
- 14.2 Protótipos: Listar as funções vazias para mostrar o planejamento: `void moverHeroi(CaminhoNode** heroi);`, `void inserirInimigo(CombateFila* fila);`.
- 14.3 Estrutura de Arquivos: Mostrar a árvore de diretórios do projeto (ex: `/src`, `/include`, `main.c`, `combate.c`, `mapa.c`).

15 Prompt Zero

- 15.0 O Prompt Zero: Se você usou IA (como eu) para iniciar a estrutura do projeto, coloque aqui o texto do primeiro prompt usado para gerar o esqueleto (muitos professores exigem isso hoje para fins de transparência).
- 16.2 Diagramas: Desenhos simples (podem ser caixas com setas) mostrando como as caixinhas dos nós se apontam na memória para formar a pista circular e o inventário.

Código do diagrama:

@startuml

title Diagrama de Estruturas de Dados do Jogo

```
class LoopNode {  
  
    +terreno : String  
  
    +proximo : LoopNode  
  
    +anterior : LoopNode  
  
}
```

```
class InventoryItem {  
  
    +nome : String  
  
    +tipo : String  
  
    +raridade : String  
  
    +atributo : int  
  
    +proximo : InventoryItem  
  
}
```

```
class MenuStack {  
  
    +push()  
  
    +pop()  
  
    +top()  
  
}
```

```
class CombatQueue {
```

```
    +enqueue()
```

```
    +dequeue()
```

```
}
```

```
class Player {
```

```
    +vida : int
```

```
    +ataque : int
```

```
    +defesa : int
```

```
}
```

```
class Enemy {
```

```
    +nome : String
```

```
    +vida : int
```

```
}
```

LoopNode --> LoopNode : próximo/anterior

InventoryItem --> InventoryItem : próximo

Player --> InventoryItem : usa itens

Player --> CombatQueue : participa

Enemy --> CombatQueue : participa

MenuStack --> Player : interface

Player --> LoopNode : percorre mapa

@enduml

Código do fluxograma:

@startuml

title Fluxograma do Game Loop Principal

start

:Iniciar Jogo;

:Carregar Save;

:Movimentação no Loop;

:Verificar Eventos;

if (Encontrou inimigo?) then (Sim)

:Iniciar Combate;

if (Jogador venceu?) then (Sim)

:Receber Loot;

:Atualizar Inventário;

:Construir Estruturas;

:Atualizar Status;

:Continuar Jornada;

else (Não)

:Game Over;

stop

endif

else (Não)

:Continuar Exploração;

endif

repeat

:Movimentação Contínua;

repeat while (Jogador Vivo?)

stop

@enduml